AAKASH VIJETA

Lucknow, India | +91 9005137180 | aakashvijeta2@gmail.com
linkedin.com/in/aakash-vijeta-1bb42b2b9 | github.com/AakashVijeta | aakashvijeta.me

SUMMARY

Data Science and AI undergraduate at IIT Guwahati who owns the full loop: reconcile messy data, design falsifiable tests against baselines, build the model, and ship it as a live product. Built and deployed LLM-integrated applications and ML systems that measure their own accuracy in production, evaluated on held-out data. Uses AI tools daily to build faster without cutting corners on statistical rigor.

EDUCATION

Indian Institute of Technology Guwahati Sep 2024 – Aug 2028 (Expected)
B.Sc. (Hons.) in Data Science and Artificial Intelligence
Relevant Coursework: Machine Learning Fundamentals, Statistical Inferencing, Time Series Analysis and Forecasting, Recommender Systems, Optimization, Linear Algebra, Data Mining and Warehousing, Relational Database Management Systems, Social Media Tools and Techniques, AI Basics, Data Structures and Algorithms

TECHNICAL SKILLS

Statistics and Experimentation: Experiment Design, Hypothesis Testing, Baseline Benchmarking, Walk-Forward Validation, Backtesting, Probability Calibration, Statistical Inference, Model Evaluation (ROC AUC, Average Precision, Brier Score)
Machine Learning: scikit-learn, XGBoost, LightGBM, Optuna, pandas, NumPy, Feature Engineering, Stable-Baselines3 (DQN, SAC), Gymnasium
AI and LLMs: LLM API Integration (Groq, OpenRouter), Llama 3.3, Provider Fallback Routing, AI-Assisted Development Workflows
Data and Backend: Python, SQL, PostgreSQL, SQLite, FastAPI, REST APIs, httpx, Data Pipelines, Multi-Source Data Reconciliation, Caching
Frontend and Deployment: JavaScript, React, Vite, Git, GitHub Actions (CI), pytest, TensorBoard, Render, Vercel, Netlify (Edge Functions)

PROJECTS

F1 Podium Predictor Python, scikit-learn, Optuna, FastAPI, PostgreSQL, React
Live: f1.aakashvijeta.me | API docs: api.aakashvijeta.me/docs

- Owned the full loop, from data ingestion through modelling, API and frontend, to a live application that predicts the top-3 finishers of every Formula 1 Grand Prix as soon as qualifying data is published.
- Built a self-evaluation system that scores every prediction against actual results across the season, and tested the model against a grid-order baseline to separate real signal from noise.
- Trained a Gradient Boosting classifier with isotonic calibration and Optuna tuning, using time-decay sample weights to adapt to the 2026 regulation reset; ROC AUC 0.92 and Brier score 0.071 on the 2026 holdout.
- Reconciled race data from three sources (PostgreSQL store, Jolpica API and FastF1) with ordered fallbacks, retries and TTL caching; built leakage-safe rolling features that shift by one race.

NiftyEdge – Swing Trading Signal Platform Python, XGBoost, FastAPI, React, Groq
Live: niftyedge.netlify.app

- Built and shipped an end-to-end pipeline (ingestion, features, training, backtesting, API, dashboard) that flags Nifty 50 stocks likely to gain 1.5% or more within 5 trading days, using XGBoost on 12 technical features.
- Guarded against look-ahead bias with 6-month walk-forward folds (precision 0.587) and backtested a 22.4% cumulative return including transaction costs and a 2.5% take-profit / 1.0% stop-loss rule.
- Integrated an LLM analyst (Llama 3.3 70B on Groq, with OpenRouter fallback) that explains the day's signals in plain language, served with ranked signals through a cached FastAPI endpoint and a React dashboard on Netlify.

Racing Line Optimization with Reinforcement Learning Python, Stable-Baselines3 (SAC), Gymnasium, NumPy, pytest
Code and demo: github.com/AakashVijeta/racing-line-optimization

- Trained a Soft Actor-Critic agent to drive fast laps on real F1 circuits in a custom Gymnasium environment with an F1-like vehicle model (traction circle, speed-dependent downforce, drag), built from 37 real circuits processed from GeoJSON and 400 procedurally generated tracks.
- Designed a leakage-free evaluation: Suzuka and 50 generated test tracks are never used for training or checkpoint selection. Procedural data plus domain randomization raised completion on unseen tracks from 21/50 to 48/50, at a measured median cost of about 5% in lap time.
- Diagnosed and fixed failure modes: auto-tuned entropy grew until it cancelled the progress reward (fixed with a cap), and a postponable stall penalty was discounted to a tie with crashing, so agents learned to stop before corners (fixed by re-deriving the penalty so stopping is always worse).
- Built reproducible tooling: validation-based model selection, checkpoint sweeps, per-track results CSVs, run configs tagged with the git commit, a pytest suite with GitHub Actions CI, and broadcast-style lap video export.